

Programming for AI

AI-2001

Functions- A Deeper Look

National University of Computer & Emerging Sciences – NUCES – Karachi



National University of Computer & Emerging Sciences – NUCES – Karachi

Course Code: AL-2001

Programming for AI

Contents

1. Introduction	2
2. Defining Functions	2
2.1. Defining a Custom Function	3
2.2. Returning a Result to a Function's Caller	3
2.3. What Happens When You Call a Function?	4
3. Functions with Multiple Parameters	4
3.1. Python's Built-In max and min Functions	5
4. Random Number Generation	5
4.1. Rolling a Six-Sided Die	5
4.2. Rolling a Six-Sided Die 6,000,000 Times	6
4.3. Seeding the Random-Number Generator for Reproducibility	7
5. Default Parameters Values	8
6. Default Parameters Values	8
6.1. Defining a Function with an Arbitrary Argument List	9
6.2. Passing an Iterable's Individual Elements as Function Arguments	9
7. Methods: Functions That Belong to Objects	9
8. Scope Rules	10
8.1. Local Scope	10
8.2. Global Scope	10
8.3 Accessing a Global Variable from a Function	10

1. Introduction

Experience has demonstrated that the most effective way to develop and sustain a large program is by breaking it down into smaller, more manageable components, a strategy known as divide and conquer. Leveraging existing functions as building blocks for new programs is crucial for software reusability and a significant advantage of object-oriented programming. By packaging code into functions, you can execute it from different parts of your program simply by calling the function, avoiding the need to duplicate potentially lengthy code. This approach also simplifies program modification; when a function's code is updated, all calls to that function will run the new version.

2. Defining Functions

You've used several built-in functions such as `int`, `float`, `print`, `input`, `type`, `sum`, `len`, `min`, and `max`, along with some from the statistics module like `mean`, `median`, and `mode`. Each of these functions serves a specific, well-defined purpose. You'll frequently create and use your own custom functions as well. In the next section, a function named `calcSquare` is defined, which calculates the square of its argument. The function is then called twice—first to square the integer value `7` (resulting in the integer `49`), and then to square the float value `2.5` (resulting in the float `6.25`).

```
[1] def calcSquare(number):  
    return number ** 2  
    # return number * number  
  
result = calcSquare(7)
```

```
[2] print(result)  
49
```

```
[3] result = calcSquare(2.5)  
print(result)  
6.25
```

The statements that define the function in the first example are written just once, but they can be called from various points in the program and as many times as needed. If you call the square function with a non-numeric argument like `'hello'`, it will result in a `TypeError` because the exponentiation operator (`**`) is only compatible with numeric values.

2.1. Defining a Custom Function

A function definition, such as the `calcSquare` function in snippet [1], starts with the `def` keyword, followed by the function's name (`calcSquare`), a set of parentheses, and a colon (`:`). Like variable names, function names should, by convention, start with a lowercase letter, and underscores should separate words in multiword names. The parentheses contain the function's parameter list—a comma-separated list of parameters that represent the data needed for the function to complete its task. The `calcSquare` function, for example, has a single parameter called `number`, which represents the value to be squared. If the parentheses are empty, it means the function does not require any parameters to perform its task.

The indented lines that follow the colon (`:`) make up the function's **block**, which includes an optional docstring and the statements that carry out the function's task.

2.2. Returning a Result to a Function's Caller

When a function finishes executing, it returns control to the line of code that called it. In the `calcSquare` function, the `return` statement:

```
return number ** 2
```

first calculates the square of `number`, then ends the function and sends the result back to the caller. For example, the first caller, `calcSquare(7)`, in snippet [1], returns the value `49`, which Python displays. Think of the return value, `49`, as replacing `calcSquare(7)` in the code, resulting in `49`. Similarly, the second caller, `calcSquare(2.5)`, in snippet [3], returns `6.25`.

Function calls can also be part of expressions. In the following code, `calcSquare(7)` is called, and `print` displays the result. Again, the return value `49` replaces `calcSquare(7)`, producing the output as shown.

```
print('The square of 7 is', calcSquare(7))  
output: The square of 7 is 49
```

There are two other ways to return control from a function:

- A `return` statement without an expression ends the function and **implicitly** returns `None`. In Python, `None` represents the absence of a value and evaluates to `False` in conditions.
- If a function has no `return` statement, it implicitly returns `None` after executing the last statement in its block.

Example 01:

```
[1] def dummyFunc():  
    print("Hello from Python!")  
    return  
  
result = dummyFunc()  
Hello from Python!  
[2] print(result)  
None
```

Example 02:

```
[1] def dummyFunc():  
    print("Hello from Python!")  
  
[2] print(dummyFunc())  
Hello from Python!  
None
```

2.3. What Happens When You Call a Function?

The expression `calcSquare(7)` passes the argument `7` to the `calcSquare` function's parameter `number`. The function then calculates `number ** 2` and returns the result. The parameter `number` exists only during the function call—it is created each time the function is called to hold the argument value, and it is destroyed when the function returns the result to the caller.

Although we didn't define variables within the `calcSquare` function, it is possible to do so. A function's parameters and any variables defined in its block are considered **local variables**—they are only accessible inside the function and exist only while the function is running. Attempting to use a local variable outside of its function results in a **NameError**, meaning the **variable is not defined**.

3. Functions with Multiple Parameters

Let's create a function that identifies and returns the largest of three values. In the next session, the function will be called three times using integers, floating-point numbers, and strings.

Example:

```
[1] def maximum(value1, value2, value3):  
    max_value = value1  
    if value2 > max_value:  
        max_value = value2  
    if value3 > max_value:  
        max_value = value3  
    return max_value  
  
[2] maximum(12, 27, 36)  
36
```

```
[3] maximum(12.3, 45.6, 9.7)
45.6
[4] maximum('yellow', 'red', 'orange')
yellow
```

We avoided adding blank lines before and after the **if** statements, as pressing enter on an empty line in interactive mode finishes the function definition. You can also call the **maximum** function with mixed types like integers and floats.

```
[5] maximum(13.5, -3, 7)
13.5
[4] maximum('yellow', 'red', 'orange')
yellow
```

3.1. Python's Built-In max and min Functions

Python already provides the functionality needed for many typical tasks. For instance, the built-in **max** and **min** functions can identify the largest and smallest values from two or more arguments, respectively.

```
[6] max('yellow', 'red', 'orange', 'blue', 'green')
yellow
[7] min(15, 9, 27, 14)
9
```

These functions can also accept an iterable argument, such as a list or a string. Utilizing built-in functions or those from the Python Standard Library modules, instead of creating your own, can save development time while enhancing program reliability, portability, and performance.

4. Random Number Generation

Let's take a short detour into a common type of programming application—simulation and game development. You can incorporate randomness by using the **random** module from the Python Standard Library.

4.1. Rolling a Six-Sided Die

Let's generate 10 random integers between 1 and 6 to simulate rolling a six-sided die.

```
[1] import random
[2] for roll in range(10):
    print(random.randrange(1, 7), end= " ")
4 3 3 2 6 5 1 2 5 5
```

We start by importing the **random** module to access its features. The **randrange** function generates an integer from the first argument up to, but not including, the second argument.

You can use the up-arrow key to recall the for loop and press Enter to run it again. You'll notice that new values appear each time.

```
[3] for roll in range(10):  
    print(random.randrange(1, 7), end=' ')
```

5 5 2 4 1 2 6 5 4 4

There are times when you might need to ensure that a random sequence can be reproduced, such as during debugging. Later in this section, we'll explain how to achieve this using the **seed** function from the **random** module.

4.2. Rolling a Six-Sided Die 6,000,000 Times

If **randrange** generates integers randomly, each number in its range should have an equal chance of being returned with every call. To demonstrate that the die faces from **1** to **6** appear with equal probability, the following script simulates **6,000,000** die rolls. When you run the script, each die face should appear around **1,000,000** times, as shown in the example output.

```
[1] import random
```

```
[2] frequency1 = 0  
frequency2 = 0  
frequency3 = 0  
frequency4 = 0  
frequency5 = 0  
frequency6 = 0
```

```
[3] for roll in range(6_000_000):  
    face = random.randrange(1, 7)  
  
    if face == 1:  
        frequency1 +=1  
    elif face == 2:  
        frequency2 +=1  
    elif face == 3:  
        frequency3 +=1  
    elif face == 4:  
        frequency4 +=1  
    elif face == 5:  
        frequency5 +=1  
    elif face == 6:  
        frequency6 +=1  
  
print(f'Face{"Frequency":>13}')  
print(f'{1:>4}{frequency1:>13}')  
print(f'{2:>4}{frequency2:>13}')  
print(f'{3:>4}{frequency3:>13}')  
print(f'{4:>4}{frequency4:>13}')  
print(f'{5:>4}{frequency5:>13}')  
print(f'{6:>4}{frequency6:>13}')
```


Face	Frequency
1	1001455
2	999068
3	999670
4	1000776
5	1000543
6	998488

The script employs nested control statements, with an **if-elif** structure inside a **for** loop, to count how often each die face appears. The for loop runs **6,000,000** times, and we used Python's underscore (**_**) as a digit separator to make this number easier to read. Writing **range(6,000,000)** would be incorrect since Python treats commas as argument separators in function calls, interpreting it as three arguments: **6**, **0**, and **0**.

For each **roll**, the script increments the corresponding counter variable by **1**. Run the program and observe the output, which may take a few seconds to finish. Each run will produce different results.

4.3. Seeding the Random-Number Generator for Reproducibility

The **randrange** function generates pseudorandom numbers based on an internal calculation that starts with a numeric value called a **seed**. Each time you start a new interactive session or run a script that uses functions from the **random** module, Python uses a different seed internally, resulting in a sequence of numbers that appear random. When debugging programs that use randomly generated data, it can be useful to generate the same sequence of numbers until you've resolved any logic errors, before testing with different values.

To achieve this, you can manually set the seed value using the **seed** function from the **random** module. This will ensure that **randrange** starts its pseudorandom number sequence from the specified seed. In the following session, snippets **[5]** and **[8]** produce identical results because snippets **[4]** and **[7]** use the same seed value (**32**).

[4]	<code>random.seed(32)</code>
[5]	<pre>for roll in range(10): print(random.randrange(1, 7), end=' ') 1 2 2 3 6 2 4 1 6 1</pre>
[6]	<pre>for roll in range(10): print(random.randrange(1, 7), end=' ') 1 3 5 3 1 5 6 4 3 5</pre>
[7]	<code>random.seed(32)</code>
[8]	<pre>for roll in range(10): print(random.randrange(1, 7), end=' ') 1 2 2 3 6 2 4 1 6 1</pre>

Snippet **[6]** produces different values because it continues the pseudorandom number sequence initiated in snippet **[5]**.

5. Default Parameters Values

When defining a function, you can set default values for its parameters. If you call the function without providing an argument for a parameter that has a default value, the function will automatically use the default value. Let's define a function **rectangle_area** with default parameter values:

```
[1] def rectangle_area(length=2, width=3):  
    return length * width
```

To set a default value for a parameter, write the parameter's name followed by an = and the default value. In this example, the default values for **length** and **width** are **2** and **3**, respectively. Parameters with default values must be listed after those without defaults in the parameter list.

The following call to **rectangle_area** includes no arguments, so Python uses both default values, as if you had called **rectangle_area(2, 3)**.

```
[2] rectangle_area()
```

6

The following call to **rectangle_area** includes only one argument. Arguments are assigned to parameters from left to right, so **10** is used for the **length**. The interpreter uses the default value of **3** for the **width**, effectively treating the call as if you had used **rectangle_area(10, 3)**.

```
[3] rectangle_area(10)
```

30

In the following call to **rectangle_area**, arguments are provided for both **length** and **width**, so IPython disregards the default values for these parameters.

```
[4] rectangle_area(10, 5)
```

50

6. Default Parameters Values

Functions that accept arbitrary numbers of arguments, like the built-in **min** and **max** functions, can handle any number of inputs. For example, take a look at the following **min** function call:

```
[1] min(88, 75, 96, 55, 83)
```

55

The function's documentation indicates that **min** has two mandatory parameters (named **arg1** and **arg2**) and an optional third parameter denoted as ***args**, which allows the function to accept any number of extra arguments. The asterisk (*) before the parameter name tells Python to gather any additional arguments into a tuple assigned to **args**. In the example call, **arg1** is assigned the value **88**, **arg2** gets **75**, and **args** receives the tuple **(96, 55, 83)**.

6.1. Defining a Function with an Arbitrary Argument List

Let's create a function named **average** that can accept any number of arguments:

```
[1] def average(*args):  
    return sum(args) / len(args)
```

55

The parameter name **args** is commonly used by convention, but you can choose any name you prefer. If the function has more than one parameter, the ***args** parameter must be placed at the end of the list. Now, let's call the **average** function multiple times with varying numbers of arguments:

```
[2] average(5, 10)
```

7.5

```
[3] average(5, 10, 15)
```

10.0

```
[4] average(5, 10, 15, 20)
```

12.5

To compute the average, divide the sum of the elements in the **args** tuple (obtained using the built-in **sum** function) by the number of elements in the tuple (found using the built-in **len** function). Note that in our **average** function definition, a **ZeroDivisionError** will occur if the length of **args** is 0.

6.2. Passing an Iterable's Individual Elements as Function Arguments

You can unpack elements from a tuple, list, or other iterable to pass them as separate arguments to a function. The ***** operator, when used with an iterable in a function call, unpacks its elements. The following code creates a list of five elements named **grades** and then uses the ***grades** expression to unpack these elements as arguments for the **average** function:

```
[5] grades = [88, 75, 96, 55, 83]
```

```
[6] average(*grades)
```

79.4

The call shown above is equivalent to **average(88, 75, 96, 55, 83)**.

7. Methods: Functions That Belong to Objects

A method is essentially a function that you invoke on an object in the format:

```
object_name.method_name(arguments)
```

For instance, in the following example, a string variable **s** is created and assigned the value **'hello'**. Next, the object's **lower** and **upper** methods are called, which generate new strings in all-lowercase and all-uppercase formats of the original string, while leaving **s** unmodified:

[1]	<code>s = 'hello'</code> <code>7.5</code>
[2]	<code>s.lower()</code> <code>'hello'</code>
[3]	<code>s.upper()</code> <code>'HELLO'</code>
[4]	<code>s</code> <code>'hello'</code>

8. Scope Rules

Each identifier in a program has a scope that dictates where it can be accessed. Within that specific part of the program, the identifier is considered to be "in scope."

8.1. Local Scope

A local variable's identifier has local scope, meaning it is "in scope" from the point of its definition until the end of the function's block. Once the function returns to its caller, the variable "goes out of scope." Therefore, local variables can only be accessed within the function where they are defined.

8.2. Global Scope

Identifiers that are defined outside of any function or class have global scope, including functions, variables, and classes. Variables with global scope are referred to as global variables. These identifiers can be accessed anywhere in a .py file or an interactive session after they have been defined.

8.3 Accessing a Global Variable from a Function

You can access a global variable's value inside a function:

[1]	<code>x = 7</code>
[2]	<code>def access_global():</code> <code> print('x printed from access_global:', x)</code>
[3]	<code>access_global()</code> <code>x printed from access_global: 7</code>

By default, you cannot directly modify a global variable within a function. When you assign a value to a variable inside a function, Python automatically creates a new local variable instead.

[4]	<code>def try_to_modify_global():</code> <code> x = 3.5</code> <code> print('x printed from try_to_modify_global: ', x)</code>
[5]	<code>try_to_modify_global()</code>

```
x printed from try_to_modify_global: 3.5
```

```
[6] x
    7
```

Within the block of the function `try_to_modify_global`, the local variable `x` hides the global `x`, making the global version inaccessible within the function's scope. Snippet [6] demonstrates that the global variable `x` remains unchanged with its original value of `7` after `try_to_modify_global` finishes executing. To modify a global variable inside a function, you need to declare it as global using the `global` statement.

```
[7] def modify_global():
    global x
    x = 'hello'
    print('x printed from modify_global: ', x)
```

```
[8] modify_global()
    x printed from modify_global: hello
```

```
[9] x
    'hello'
```

Talha Shahid

Talha Shahid